

# Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Prof. Alessandro Ricci

[module 1.1]

## CONCURRENT PROGRAMMING INTRODUCTION

# SUMMARY

- Concurrent programming
  - motivations: HW evolution
  - basic jargon
    - processes interaction, cooperation, competition,
    - mutual exclusion, synchronisation
    - problems: deadlocks, starvation, livelocks
- Concurrent languages, mechanisms, abstractions
  - overview

# CONCURRENCY AND CONCURRENT SYSTEMS

- Concurrency as a main concept of many domains and systems
  - operating systems, multi-threaded and multi-process programs, distributed systems, control systems, *real-time* systems,...
- General definitions

– “*In computer science, concurrency is a property of systems in which several computational processes are executing **at the same time**, and potentially **interacting** with each other.*”  
[ROS-97]

– “*Concurrency <sup>riguarda</sup> ~~is concerned with~~ the fundamental aspects of systems <sup>composti da</sup> of multiple, **simultaneously active** computing agents, that **interact** with one another*” [CLE-96]

## Common aspects

- systems with multiple activities or **processes** whose execution **overlaps in time**
- activities can have some kind of *dependencies*, therefore can **interact**

# CONCURRENT PROGRAMMING

- **Concurrent programming**
  - building programs in which multiple computational activities overlap in time and typically interact in some way
- **Concurrent program**
  - finite set of *sequential* programs that can be executed in parallel, i.e. overlapped in time
    - a sequential program specifies sequential execution of a list of statements
    - the execution of a sequential program is called **process**
    - a concurrent program specifies two or more sequential programs that may be executed concurrently as *parallel processes*
  - the execution of a concurrent program is called *concurrent computation* or *elaboration*



# TERMINOLOGY

## Concurrent programming

La concorrenza è un problema di struttura (voglio che il mio codice sia organizzato bene, reattivo e pulito).

- building programs in which multiple computational activities *overlap* in time and typically interact in some way
  - without necessarily running on separate physical processors
- logical/abstract/programming level
- focus on program organization

la concorrenza è un'astrazione mentale e concettuale. Quando scrivi codice concorrente, stai ragionando sulla logica del programma, non sull'hardware.

Scrivere codice concorrente serve prima di tutto a disaccoppiare le varie responsabilità del software, rendendolo più pulito e gestibile.

## Parallel programming

Il parallelismo è un problema di performance (voglio che il codice vada più veloce sfruttando la potenza del silicio).

- the execution of programs overlaps in time by running on *separate physical processors*
- physical level
- focus on performance

## Distributed programming

- when processors are distributed over a network
- no shared memory

- Concorrenza: Riguarda il modo in cui un programma è strutturato. Significa scomporre un problema in parti che possono essere gestite separatamente. In un sistema concorrente, più attività avanzano insieme perché il sistema passa rapidamente da una all'altra (tramite lo scheduling del sistema operativo), dando l'illusione della contemporaneità anche se la macchina ha un solo processore.

- Parallelismo: Riguarda il modo in cui il programma viene eseguito. È un caso specifico (e un sottoinsieme) della concorrenza che richiede necessariamente un hardware con più processori o core. In questo caso, le attività non si alternano, ma accadono letteralmente nello stesso identico istante.

- In sintesi: La concorrenza è l'abilità di gestire (comporre) molte cose contemporaneamente. Il parallelismo è l'abilità di fare molte cose contemporaneamente.

# CONCURRENT VS. PARALLEL

- Rob Pyke's talk about "Go Concurrency Patterns" [\*]
  - world as set of interacting agents
    - this is not captured by sequential programming paradigms
  - *La concorrenza è la strutturazione di un programma come composizione di processi che eseguono in modo indipendente.*  
concurrency as the composition of independently executing computations.
  - it's a way of *structuring software*, to write clean code that interacts well with the real world.
  - it's a model for software construction
    - easy to understand, use, reason about
  - *It is not parallelism. it enables parallelism*
    - parallelism => algorithmic view, performance

Significa che se progetti il tuo software in modo concorrente (spezzettandolo in compiti indipendenti che comunicano tra loro), il computer potrà decidere di eseguirli in parallelo se ha più processori a disposizione. Se invece il tuo codice è un unico blocco sequenziale, non potrà mai essere parallelo, a prescindere dalla potenza del computer.

(\*) <https://www.youtube.com/watch?v=f6kdp27TYZs>

# CONCURRENCY “PARADIGMS”

- **Multi-threaded programming**

- shared state
- synchronisation mechanisms
  - semaphores, monitors

- **Message-based programming**

- no shared state
- interaction by means of message exchange
- *asynchronous vs. synchronous*

- **Event-driven programming**

- the flow of the program is determined by events
  - user actions (mouse clicks, key presses), sensors, messages from other threads/process/apps

La programmazione concorrente è il paradigma ombrello (il "concetto generale") che definisce l'approccio alla progettazione di software in cui un sistema è composto da più flussi di esecuzione indipendenti, che devono coordinarsi, interagire o condividere risorse.

Mentre la concorrenza identifica il problema architetturale e il modello mentale (come dividere il lavoro in parti indipendenti), i 5 paradigmi che hai elencato sono le diverse strategie di astrazione e implementazione utilizzate per realizzare ed esprimere quella concorrenza nel codice.

Qual è la differenza?

- Programmazione Concorrente è l'idea astratta di strutturare un programma come un insieme di task/agenti autonomi che procedono insieme, gestendo l'ordine degli eventi, l'accesso alle risorse e lo scambio di informazioni.  
- I Paradigmi sono le diverse astrazioni offerte dai linguaggi/framework per gestire il coordinamento dei flussi senza impazzire con race condition, deadlock o complessità architetturale.

# CONCURRENCY “PARADIGMS”

- **Asynchronous programming**

progettazione di programmi che utilizzano

- designing programs featuring asynchronous actions and requests
  - never blocking dogma
  - future mechanisms, callbacks

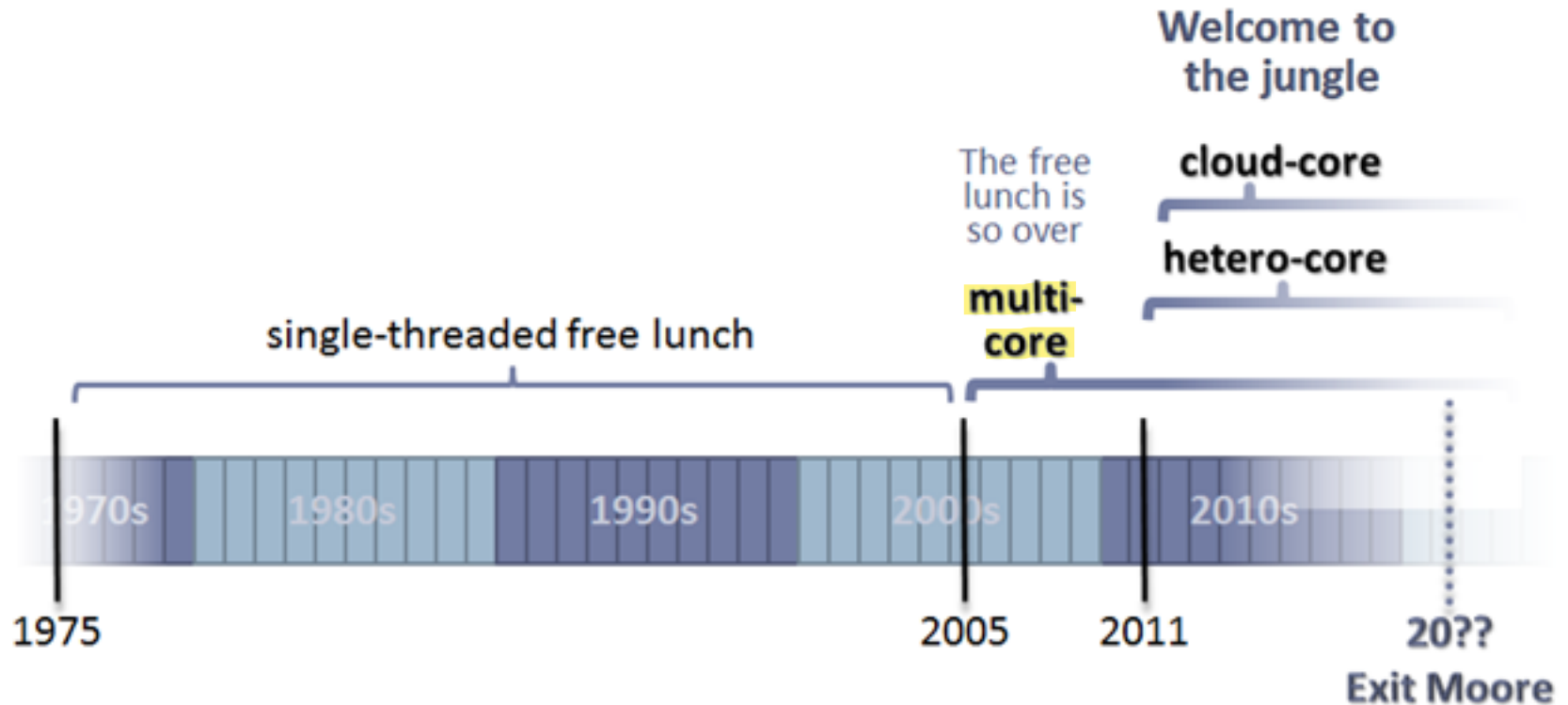
- **Reactive programming**

- the flow of the program is designed around data flows and the propagation of change

# PARALLEL COMPUTERS:

## “THE HARDWARE (CORE) JUNGLE”

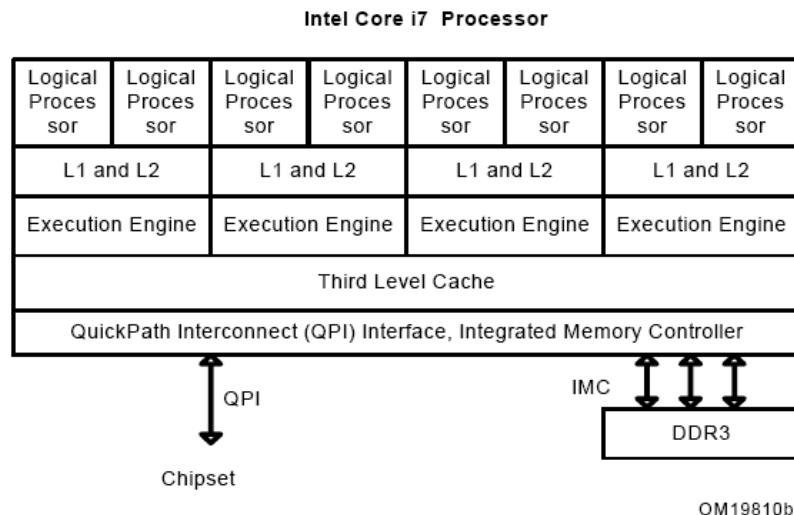
- *“The Free Lunch is Over. Now Welcome to the Hardware Jungle”*  
(Herber Sutter, [SUT-12])



# MULTI-CORE ARCHITECTURES

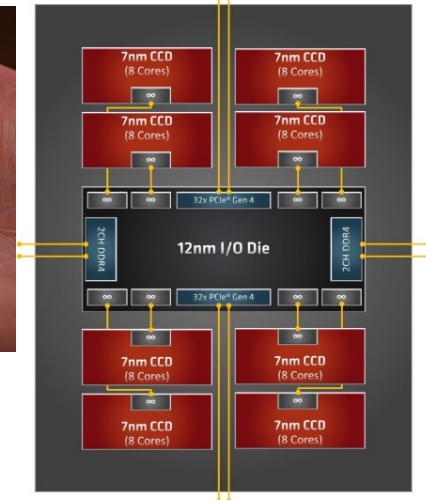
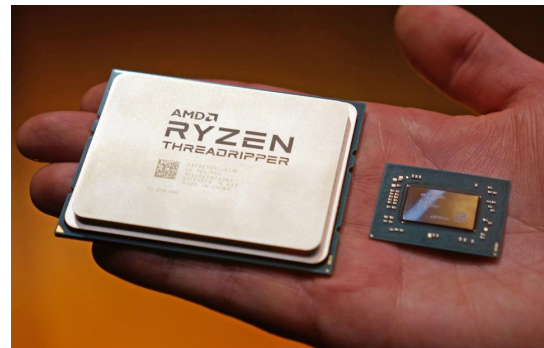
- **Chip multiprocessors - Multicore**
  - multiple cores on a single chip
    - sharing RAM, possibly sharing cache levels
    - examples: Intel Core i7, AMD Rizen Threadripper 3990X

Intel Core i7 family, ~2009



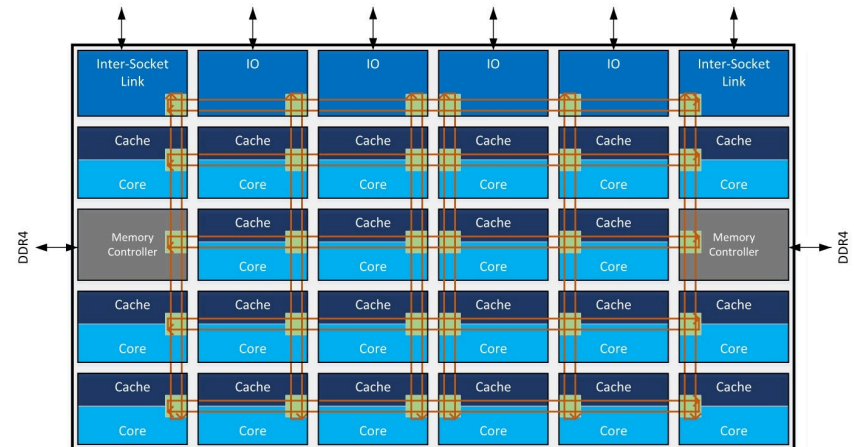
AMD Ryzen™ Threadripper™ 3990X Processor (2020)

- 64-bit **64-core** high-performance x86 desktop microprocessor
- Base frequency of 2.9 GHz with a TDP of 280 W and a boost of up to 4.3 GHz.
- Supports up to 512 GiB of quad-channel DDR4-3200 memory.



# MULTI-CORE HYBRID ARCHITECTURES

- Recent architecture are *hybrid*, combining CPU cores of varying sizes
  - Performance-cores (P-cores)**
    - traditional design, featuring high clock speeds
  - Efficient-cores (E-cores)**
    - slower but physically smaller, consume much less power
- Example:
  - Intel Core i9 - 14th Gen Intel® Core™ desktop processors - 24 cores (up to 32 threads)
    - 8 P-cores
      - highest-performing CPU core, 5.8 GHz
    - 16 E-cores
      - for background tasks



# MULTI-CORE HYBRID ARCHITECTURES

- Another example: **AMD Zen Core Architecture - last gen: Zen 5**
  - Ryzen (consumer) and EPYC (server) series
  - scalability and high multi-core performance
- Key Aspects of Modern AMD Architecture: Chiplet Design
  - instead of one large silicon die, AMD uses "chiplets" (Core Complex Dies or CCDs) connected by a high-speed fabric, allowing for higher core counts
    - up to 16 cores/32 threads on desktop, 128+ on server
- In a die we can have either P-Cores or E-Cores
  - “Zen 5” — core is optimized for high performance
  - “Zen 5c” — core optimized for density and efficiency



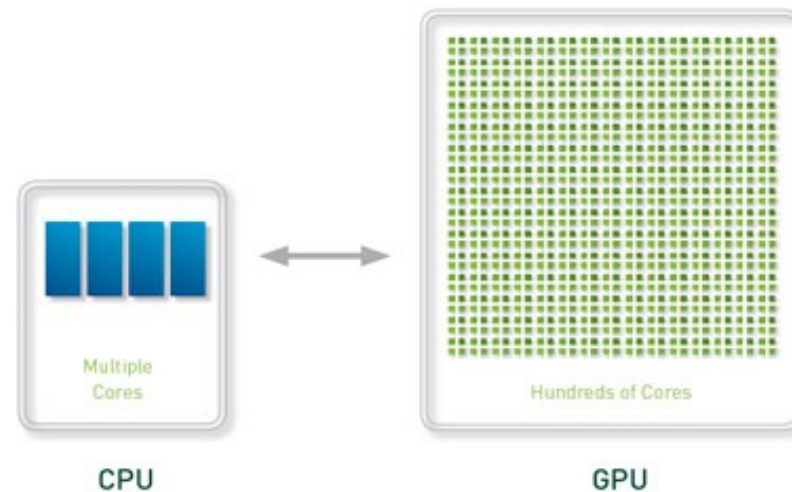
# PARALLEL COMPUTERS: HETEROGENEOUS CORES & MANY-CORE

## Heterogeneous Chips Designs

- augmenting a standard processor with one or more specialised compute engines, called attached processors
  - examples: Graphical Processing Units (GPU), **GPGPU** (General-Purpose Computation on Graphics Hardware), Field-Programmable Gate Array (FPGA), Cell processors, **CUDA** architecture

Quando si parla di Chip Design, ci si riferisce alla progettazione del singolo pezzo di silicio (o di un unico pacchetto integrato).

L'Heterogeneous Chip Design consiste nel prendere la CPU e la GPU (o la NPU) e integrarle nello stesso silicio o nello stesso minuscolo contenitore.

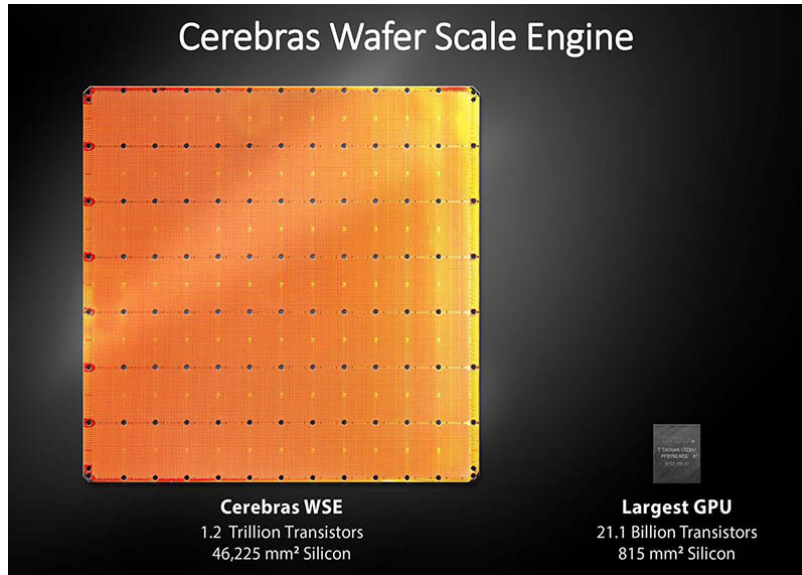


# PARALLEL COMPUTERS: HETEROGENEOUS CORES & MANY-CORE

- Example: Apple Silicon M5
  - **SoC** - including CPU, GPU, NPU (neural processing unit) used for machine learning), ISP (coprocessor for elaborating images from photo sensor), Mx (coprocessor for elaborating data from integrated sensors), SEP (data protection)
  - Up to 10 CPU cores
    - 3 or 4 P-cores and 4 or 6 E-cores
  - Up to 10 GPU cores

# PARALLEL COMPUTERS: DESIGNED FOR AI

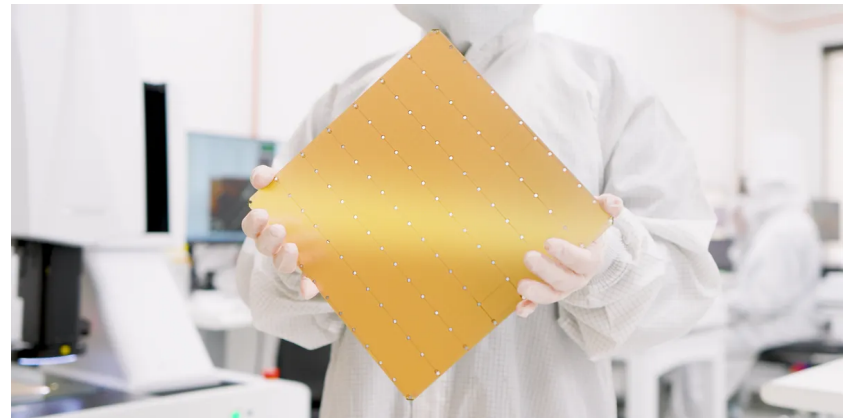
- E.g. Cerebras — latest CS-3 system (2024)



| Wafer Scale Engine-3 vs. H100 |                        |                          |                    |
|-------------------------------|------------------------|--------------------------|--------------------|
|                               | Cerebras WSE-3         | Nvidia H100              | Cerebras Advantage |
| Chip size                     | 46,225 mm <sup>2</sup> | 814 mm <sup>2</sup>      | 57x                |
| Cores                         | 900,000                | 16,896 FP32 + 528 Tensor | 52x                |
| On-chip memory                | 44 Gigabytes           | 0.05 Gigabytes           | 880x               |
| Memory bandwidth              | 21 Petabytes/sec       | 0.003 Petabytes/sec      | 7,000X             |
| Fabric bandwidth              | 214 Petabits/sec       | 0.0576 Petabits/sec      | 3,715X             |



PCD LM - ISI - Cesena - UNIBO



Concurrent Programming Overview

# PARALLEL COMPUTERS: SUPER-COMPUTERS

- Traditionally used by national labs and large companies
- Different kind of architectures, including clusters
- Typically large number of processors (multi-core) connected by ad-hoc networks
  - example: Fugaku Super computer

RIKEN Center for Computational Science

Manufacturer: Fujitsu

Cores: 7,630,848

Memory: 5,087,232 GB

Processor: A64FX 48C 2.2GHz

Interconnect: Tofu interconnect D



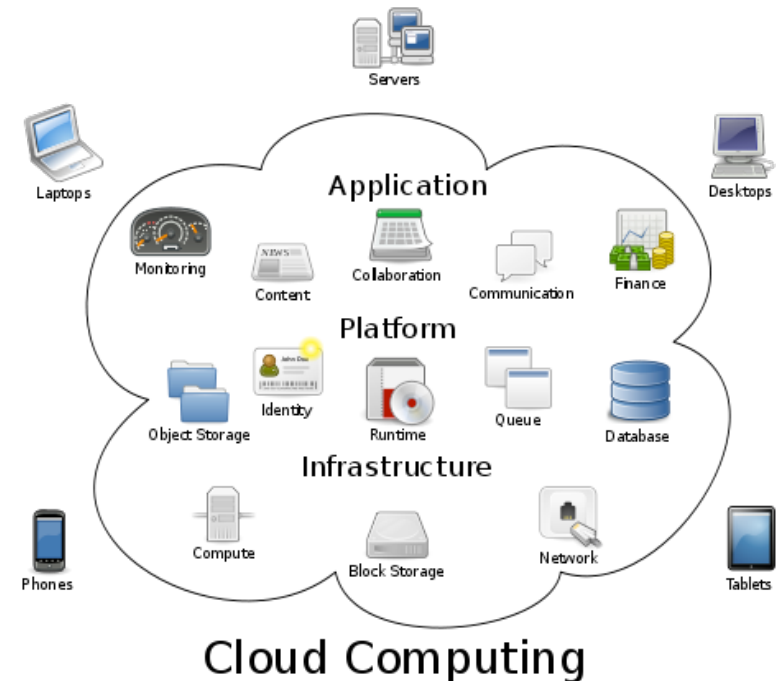
# PARALLEL COMPUTERS: CLUSTERS / GRID

- Made from commodity parts
  - nodes are boards containing one or few processors, RAM and sometimes a disk storage
  - nodes connected by commodity interconnect
    - e.g. Gigabit Ethernet, Myrinet, InfiniBand, Fiber Channel
- Memory not shared among the machines
  - processors communicate by message passing
- Examples
  - System X supercomputer at Virginia Tech, a 12.25 TFlops computer cluster of 1100 Apple XServe G5 2.3 GHz dual-processor machines (4 GB RAM, 80 GB SATA HD) running Mac OS X and using InfiniBand interconnect
- Grid computing



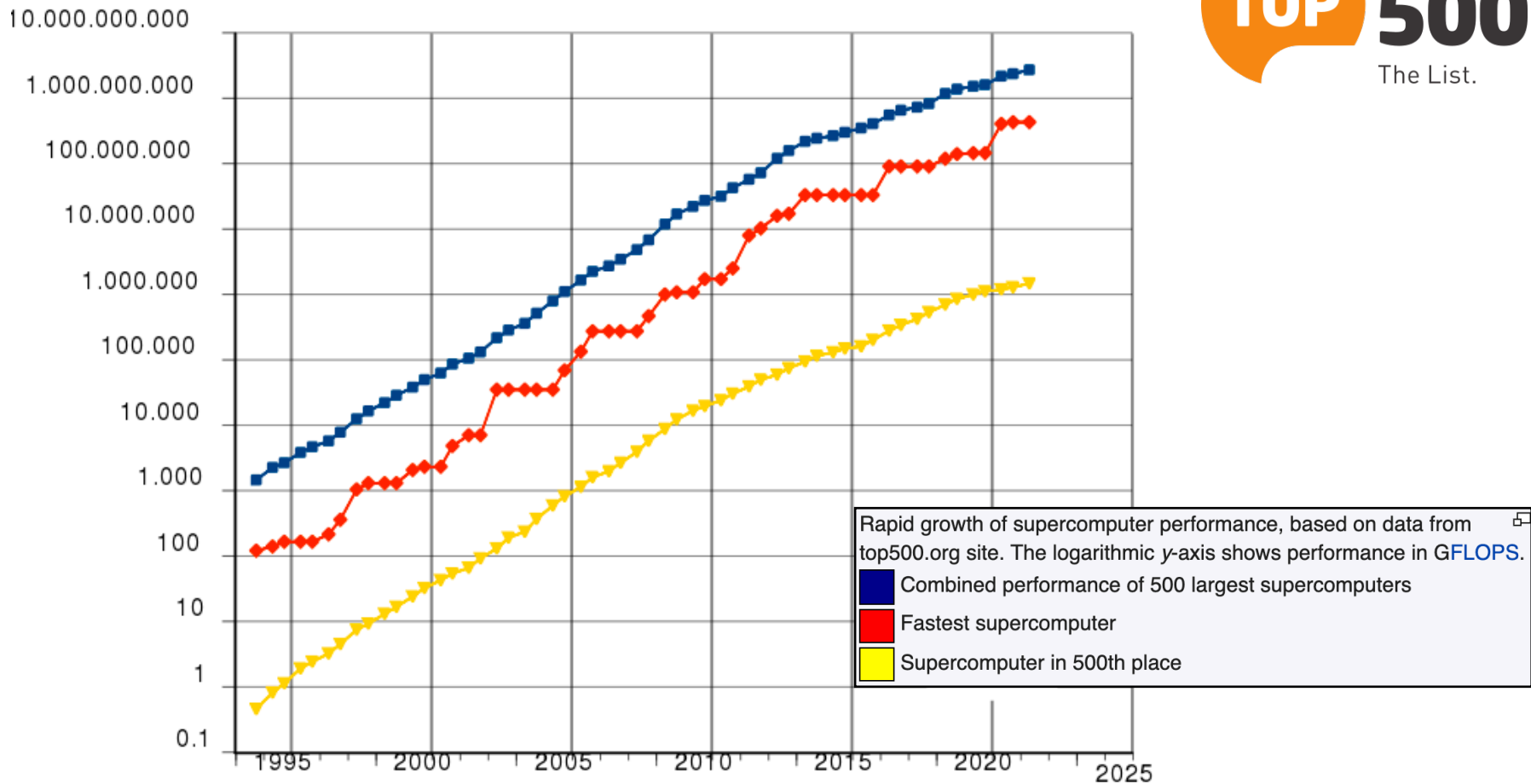
# PARALLEL COMPUTERS: CLOUD COMPUTING

- Delivering computing **as a service** through the network
  - shared resources, software, and information are provided to computers and other devices as a metered service over a network (typically the Internet)
- X as a Service
  - Software as a Service (SAAS)
  - Platform as a Service (PAAS)
  - Infrastructure as a Service (IAAS)
- Public clouds, private clouds
- Examples
  - Amazon EC2 (Elastic Computing Cloud)
  - Microsoft Azure, Google App Engine



# TOP 500 - FASTEST SUPERCOMPUTERS

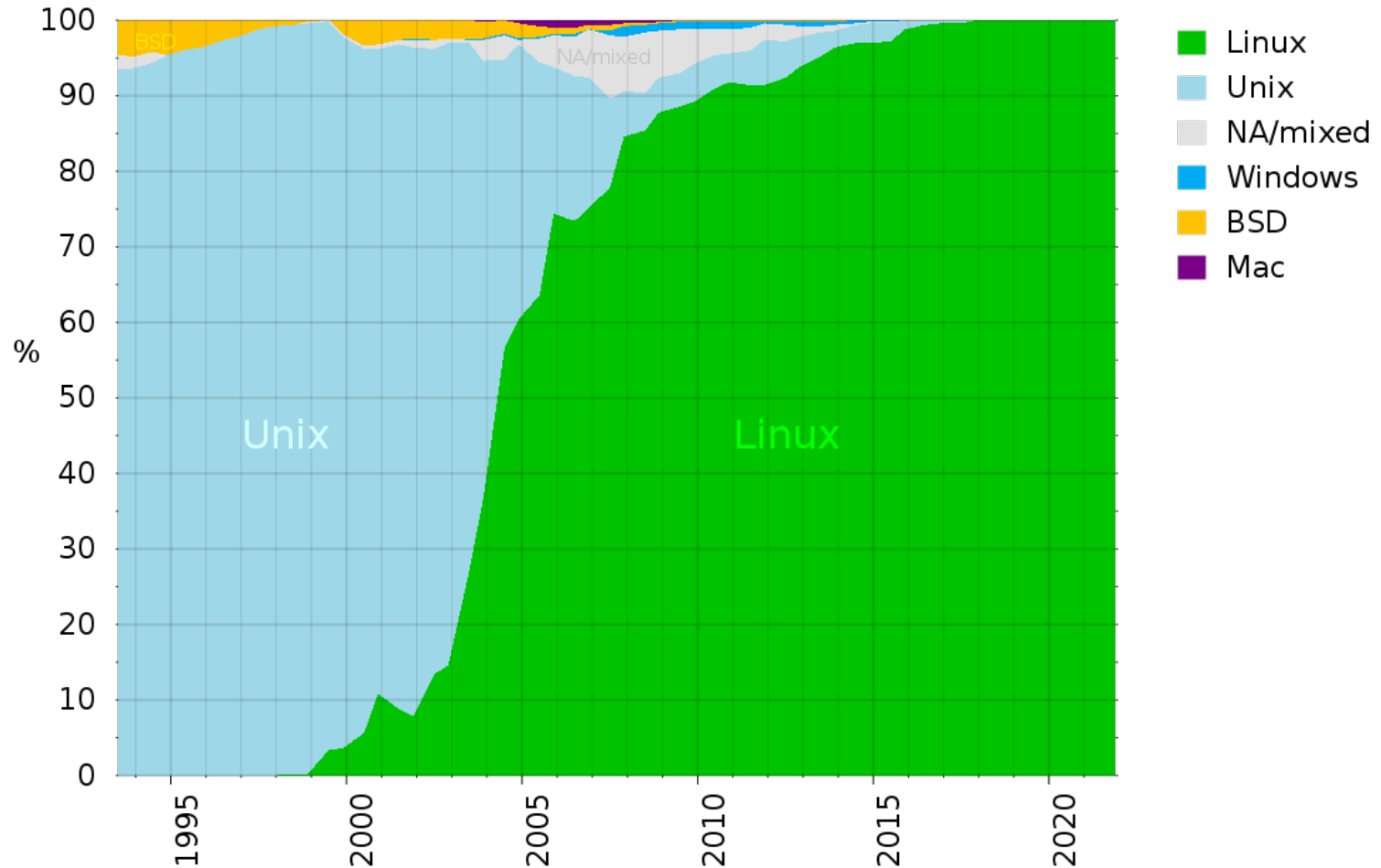
- <https://en.wikipedia.org/wiki/TOP500>





# TOP 500 - FASTEST SUPERCOMPUTERS

- OS kernels used in supercomputers



<https://en.wikipedia.org/wiki/TOP500>



# THE FASTEST (Nov. 2025)

<https://en.wikipedia.org/wiki/TOP500>

Top 10 positions of the total 10,000 in November 2025

| Rank<br>(previous) | Rmax<br>Rpeak<br>(PetaFLOPS) | Name          | Model                 | CPU<br>cores                                                                                 | Accelerator<br>(e.g. GPU)<br>cores                        | Total cores<br>(CPUs +<br>accelerators) | Interconnect                                | Manufacturer | Site<br>country                                                        | Year | Operating<br>system                                        |
|--------------------|------------------------------|---------------|-----------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------|---------------------------------------------|--------------|------------------------------------------------------------------------|------|------------------------------------------------------------|
| 1 ▲                | 1,809.00<br>2,821.10         | El<br>Capitan | HPE Cray<br>EX255a    | 1,080,000<br>(45,000 ×<br>24-core<br>Optimized<br>4th<br>Generation<br>EPYC 24C<br>@1.8 GHz) | 10,260,000<br>(45,000 ×<br>228 AMD<br>Instinct<br>MI300A) | 11,340,000                              | Slingshot-11                                | HPE          | Lawrence<br>Livermore<br>National<br>Laboratory<br>🇺🇸 United<br>States | 2024 | Linux<br>(TOSS)                                            |
| 2                  | 1,353.00<br>2,055.72         | Frontier      | HPE Cray<br>EX235a    | 614,656<br>(9,604 ×<br>64-core<br>Optimized<br>3rd<br>Generation<br>EPYC 64C<br>@2.0 GHz)    | 8,451,520<br>(38,416 ×<br>220 AMD<br>Instinct<br>MI250X)  | 9,066,176                               | Slingshot-11                                | HPE          | Oak Ridge<br>National<br>Laboratory<br>🇺🇸 United<br>States             | 2022 | Linux<br>(HPE<br>Cray OS)                                  |
| 3                  | 1,012.00<br>1,980.01         | Aurora        | HPE Cray EX           | 1,104,896<br>(21,248 ×<br>52-core<br>Intel Xeon<br>Max 9470<br>@2.4 GHz)                     | 8,159,232<br>(63,744 ×<br>128 Intel<br>Max 1550)          | 9,264,128                               | Slingshot-11                                | HPE          | Argonne<br>National<br>Laboratory<br>🇺🇸 United<br>States               | 2023 | Linux<br>(SUSE<br>Linux<br>Enterprise<br>Server 15<br>SP4) |
| 4 ▲                | 1,000.00<br>1,226.28         | JUPITER       | BullSequana<br>XH3000 | 1,694,592<br>(23,536 ×<br>72-Arm<br>Neoverse<br>V2 cores<br>Nvidia<br>Grace<br>@3 GHz)       | 3,106,752<br>(23,536 ×<br>132 Nvidia<br>Hopper<br>H100)   | 4,801,344                               | Quad-rail<br>NVIDIA<br>NDR200<br>Infiniband | Atos         | EuroHPC JU<br>🇪🇺 European<br>Union, Jülich,<br>🇩🇪 Germany              | 2025 | Linux<br>(RHEL)                                            |
| 5                  | 561.20<br>846.84             | Eagle         | Microsoft NDv5        | 172,800<br>(3,600 ×<br>48-core<br>Intel Xeon<br>Platinum<br>8480C<br>@2.0 GHz)               | 1,900,800<br>(14,400 ×<br>132 Nvidia<br>Hopper<br>H100)   | 2,073,600                               | NVIDIA<br>Infiniband<br>NDR                 | Microsoft    | Microsoft<br>🇺🇸 United<br>States                                       | 2023 | Linux<br>(Ubuntu<br>22.04<br>LTS)                          |

# THE FASTEST (Nov. 2025)

<https://en.wikipedia.org/wiki/TOP500>

|    |                  |          |                      |                                                                            |                                                 |           |                                    |         |                                                          |      |                                |
|----|------------------|----------|----------------------|----------------------------------------------------------------------------|-------------------------------------------------|-----------|------------------------------------|---------|----------------------------------------------------------|------|--------------------------------|
| 6  | 477.90<br>606.97 | HPC6     | HPE Cray EX235a      | 213,120<br>(3,330 × 64-core<br>Optimized 3rd Generation EPYC 64C @2.0 GHz) | 2,930,400<br>(13,320 × 220 AMD Instinct MI250X) | 3,143,520 | Slingshot-11                       | HPE     | Eni S.p.A<br>European Union, Ferrara Erbognone,  Italy   | 2024 | Linux (RHEL 8.9)               |
| 7  | 442.01<br>537.21 | Fugaku   | Supercomputer Fugaku | 7,630,848<br>(158,976 × 48-core Fujitsu A64FX @2.2 GHz)                    | -                                               | 7,630,848 | Tofu interconnect D                | Fujitsu | Riken Center for Computational Science<br>Japan          | 2020 | Linux (RHEL)                   |
| 8  | 434.90<br>574.84 | Alps     | HPE Cray EX254n      | 748,800<br>(10,400 × 72-Arm Neoverse V2 cores Nvidia Grace @3.1 GHz)       | 1,372,800<br>(10,400 × 132 Nvidia Hopper H100)  | 2,121,600 | Slingshot-11                       | HPE     | CSCS Swiss National Supercomputing Centre<br>Switzerland | 2024 | Linux (HPE Cray OS)            |
| 9  | 379.70<br>531.51 | LUMI     | HPE Cray EX235a      | 186,624<br>(2,916 × 64-core<br>Optimized 3rd Generation EPYC 64C @2.0 GHz) | 2,566,080<br>(11,664 × 220 AMD Instinct MI250X) | 2,752,704 | Slingshot-11                       | HPE     | EuroHPC JU<br>European Union, Kajaani,  Finland          | 2022 | Linux (HPE Cray OS)            |
| 10 | 241.20<br>306.31 | Leonardo | BullSequana XH2000   | 110,592<br>(3,456 × 32-core Xeon Platinum 8358 @2.6 GHz)                   | 1,714,176<br>(15,872 × 108 Nvidia Ampere A100)  | 1,824,768 | Quad-rail NVIDIA HDR100 Infiniband | Atos    | EuroHPC JU<br>European Union, Bologna,  Italy            | 2023 | Linux (RHEL 8) <sup>[31]</sup> |

# FLYNN'S TAXONOMY

Serve a categorizzare qualsiasi architettura di sistema di computazione in base a come gestisce due elementi fondamentali: le istruzioni (il codice da eseguire) e i dati (le informazioni su cui operare).

Categorization of all computing systems according to the *number of instruction stream and data stream*

- stream as a sequence of instruction or data on which a computer operate

Four possibilities

Un'istruzione alla volta su un solo dato alla volta.

– **Single Instruction, Single Data (SISD)**

C'è un unico processore che esegue il programma in modo strettamente sequenziale. Prende un'istruzione, prende il dato associato, esegue il calcolo e passa alla successiva.

- Von-Neumann model, single processor computers

Una sola istruzione eseguita contemporaneamente su tanti dati diversi.

– **Single Instruction, Multiple Data (SIMD)**

- single instruction stream concurrently broadcasted to multiple processors, each with its own data stream

C'è un'unica unità di controllo che impartisce lo stesso identico comando (es. "Moltiplica per 2") a moltissimi processori paralleli. Ognuno di questi processori applicherà quel comando su un dato diverso presente nel proprio flusso di dati.

- fine grained parallelism, vector processors

Più istruzioni diverse eseguite contemporaneamente sullo stesso identico dato.

– **Multiple Instruction, Single Data (MISD)**

- no well known systems fit this

Più processori indipendenti che eseguono istruzioni diverse su dati diversi.

– **Multiple Instruction, Multiple Data (MIMD)**

Ogni core o processore del sistema è completamente autonomo. Ha il suo flusso di istruzioni (sta eseguendo un programma o una porzione di codice diversa) e lavora su dati separati.

- each processor has its own stream of instructions operating on its own data

# MIMD MODELS

- MIMD category can be then decomposed according to memory organization
  - **shared memory**
    - all processes (processors) share a single address space and communicate each other by writing and reading shared variables
  - **distributed memory**
    - each process (processor) has its own address space and communicate with other process by *message passing* (sending and receiving messages)

# MIMD FURTHER CLASSIFICATIONS

- Two further classes for shared-memory computers
  - **SMP (Symmetric Multi-processing Architecture)**
    - all processors share a connection to a common memory and access all location memories at equal speed
  - **NUMA (Non-uniform Memory Access)**
    - the memory is shared, by some blocks of memory may be physically more closely associated with some processors than others

# MIMD FURTHER CLASSIFICATIONS

- Two further classes for distributed-memory computers
  - **MPP (Massively Parallel Processors)**
    - processors and the network infrastructure are tightly coupled and specialized for a parallel computer
    - extremely scalable, thousands of processors in a single system
    - for High-Performance Computing (HPC) applications
  - **Clusters**
    - distributed-memory systems composed of off-the-shelf computers connected by an off-the-shelf network
    - e.g. Beowulf clusters ( = clusters on Linux)
  - **Grid**
    - systems that use distributed, heterogeneous resources connected by LAN and/or by WAN, without a common point of administration

# WHY CONCURRENT PROGRAMMING: PERFORMANCE

- **Performance improvement**

- increased application **throughput**

- by exploiting parallel hardware

Il throughput è la quantità di lavoro utile che un sistema riesce a completare in una determinata unità di tempo.

- increased application **responsiveness**

La responsiveness è la capacità del sistema di rispondere rapidamente a un input o a un evento

- by optimizing the interplay among CPU and I/O activities

l'interazione tra le attività della CPU e quelle di I/O

- Quantitative measurement for performance: **speedup**

$$S = \frac{T_1}{T_N}$$

$N$  is the number of processors

$T_1$  is the execution time of the sequential algorithm

$T_N$  is the execution time of the parallel algorithm with  $N$  processors

Si ha il massimo speedup quando suddivido il lavoro su  $N$  processori

# AMDAHL'S LAW

la Legge di Amdahl si concentra sullo Speedup e ti dice qual è lo speedup massimo teorico che puoi ottenere parallelizzando un programma, evidenziando i limiti imposti dalla parte di codice che non può essere parallelizzata.

- Maximum speedup parallelizing a system:

Se  $P=0$ ,  $S=1$   
(programma non  
parallelizzabile o  
problema di  
implementazione)

$$S = \frac{1}{1 - P + \frac{P}{N}}$$

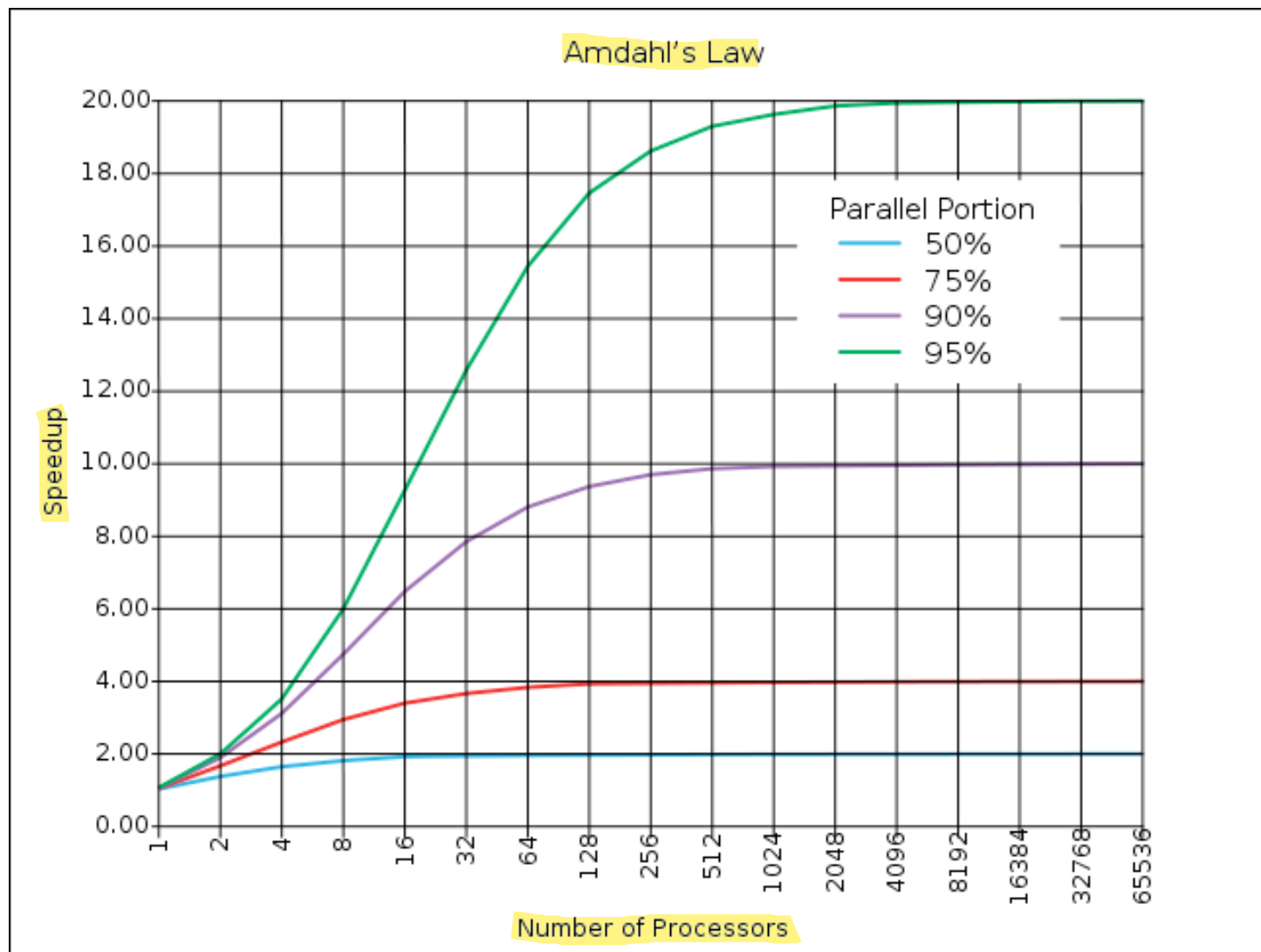
Viene raccolto  $T_1$  e si semplifica  
arrivando alla formula di Amdahl

$S = T_1 / N$  (massima suddivisione  
del lavoro su  $N$  processori)

- $P$  is the <sup>percentuale</sup> proportion of a program that can be made parallel
  - $(1-P)$  is then the part that cannot be parallelized
- <sup>In teoria</sup> Theoretically, maximum for  $P = 1$  (linear speedup) Caso Migliore:  $P=1$  allora  $S=N$ 
  - actually there are specific cases with  $S > N$  (super-linear) speedup



# AMDAHL'S LAW



# REMARKS FROM AMDAHL'S LAW

- We can improve performances by adding processors only for those parts that can run in parallel, not for sequential parts
- => strong impact on performances (of sequential parts)
- But sequential parts are often needed for correctness (next module)

Se in un programma l'95% è parallelizzabile e il 5% è sequenziale, su un sistema massivamente parallelo quel 5% sequenziale arriverà a occupare la quasi totalità del tempo di calcolo rimanente, diventando il vero e proprio collo di bottiglia del sistema.

Non si può semplicemente eliminare la parte sequenziale per fare andare il programma più veloce, perché serve a garantire che il programma funzioni bene e non perda dati. In un'applicazione parallela, le parti sequenziali sono indispensabili per:

- Inizializzazione e Configurazione: Leggere i file di configurazione, allocare la memoria o avviare le connessioni di rete prima di far partire i thread di calcolo.
- Sincronizzazione (Orchestrazione): Coordinare i vari thread affinché non creino conflitti (Race Conditions). Ad esempio, l'uso di lock, semafori o il tracciamento dei messaggi tramite clock logici per garantire l'ordine degli eventi richiedono punti di sincronizzazione intrinsecamente sequenziali.
- Aggregazione dei risultati: Raccogliere i dati elaborati separatamente dai singoli core per unirli in un unico output finale corretto o scriverli in un file di log comune.

# EFFICIENCY

L'efficienza è la misura normalizzata dello speedup, espressa come il rapporto tra lo speedup ottenuto e il numero di processori utilizzati per ottenerlo.

Normalized measure of speed-up <sup>che indica l'efficacia con cui viene utilizzato</sup> ~~indicating how effectively each~~  
<sup>ciascun processore</sup> ~~processor is used~~ (S = speedup, N = number of processors)

Mentre lo speedup ti dice solo di quante volte il programma è diventato più veloce (es. S = 8), non tiene conto di quante risorse hai dovuto spendere per raggiungere quel risultato. L'efficienza normalizza questo valore portandolo in una scala da 0 a 1:

Se un programma ha uno speedup di 8 usando 8 processori, l'efficienza è = 1. Ogni processore è sfruttato al massimo delle sue potenzialità.

Se lo stesso programma ottiene uno speedup di 8, ma usando 16 processori, l'efficienza scende a 0.5. Significa che stai sprecando metà della potenza computazionale a tua disposizione, spesso a causa dei colli di bottiglia sequenziali o dell'overhead di sincronizzazione evidenziati dalla Legge di Amdahl.

$$E = \frac{S}{N}$$

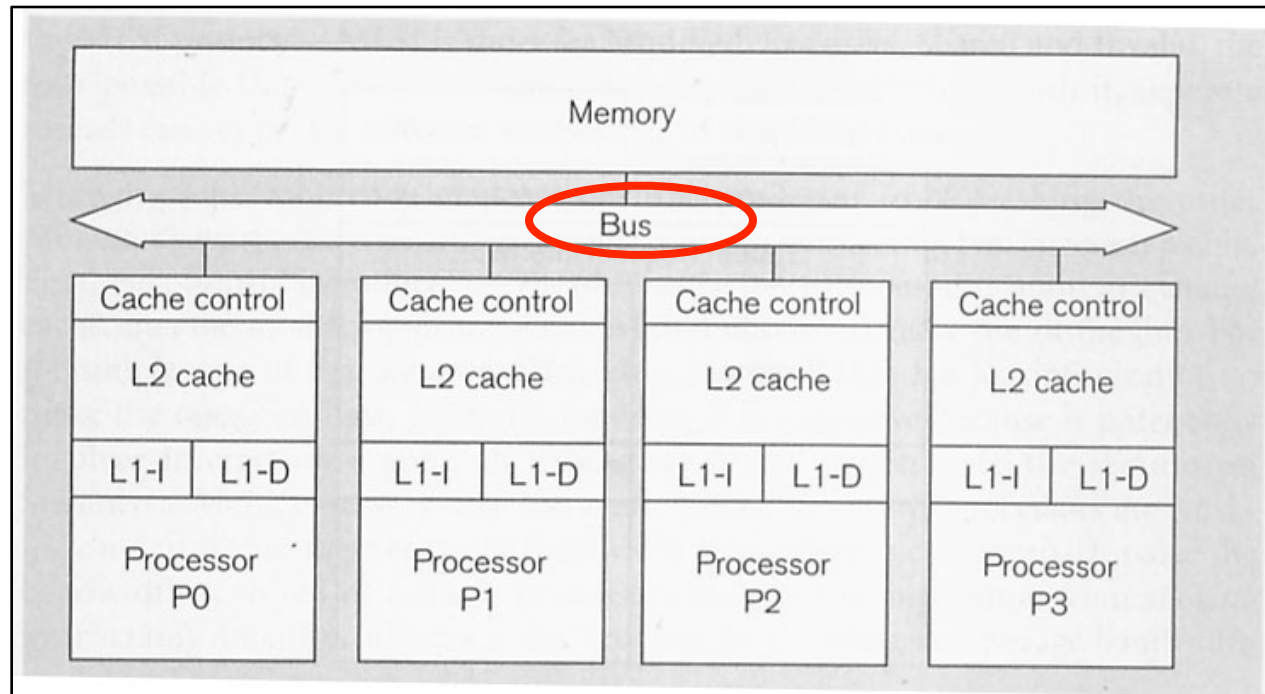
- The ideal efficiency is 1 = all processors are used at full capacity
  - typically lower

# A NEW BOTTLENECK: MEMORY

- Shared memory and bus as potential bottleneck
  - only one memory operation takes place at a time
  - importance of the cache
    - cache coherency protocol more and more complex and smart

A livello hardware classico, la memoria RAM o il bus di comunicazione principale possono gestire un solo accesso (lettura o scrittura) alla volta per ogni ciclo di clock o per canale di memoria.

La cache memorizza localmente e temporaneamente le informazioni usate più spesso dal singolo core.



# CONCURRENT PROGRAMMING AS A PARADIGM

# WHY CONCURRENT PROGRAMMING: DESIGN & ABSTRACTION

- Abstraction and engineering
  - define a proper level of abstraction for programs which interact with the environment, control multiple activities and handle multiple events..
    - objects from OOP are not enough
- Concurrency as a tool for software design and construction
  - rethinking to the way in which we solve problems
    - basic algorithms & data structures
  - rethinking to the way in which we design and build systems
    - new level of abstraction
      - different kind of decomposition, modularization, encapsulation
- Affecting the full engineering spectrum
  - **modelling, design, implementation, verification, testing**

OOP è eccellente per modellare dati e comportamenti, ma ragiona in modo intrinsecamente sequenziale (la OOP da sola non basta per modellare sistemi che devono gestire eventi asincroni e simultanei.).

# CONCEPTS: PROCESSES

- **Processes** ~ a sequential program in execution (flusso di controllo sequenziale indipendente)
  - the basic unit of a concurrent system, single thread of control
    - logical thread of control / control flow, not necessarily physical
  - sequence of instructions operating together as a group
    - unit of work (task)
  - abstract / general concept
    - ...not necessarily related to OS processes

## *speed independence*

Lo speed independence è una proprietà del sistema concorrente nel suo insieme (quindi del programma concorrente)

- process execution is meant to be completely asynchronous with each other
  - we can't do any assumption about their speed

Dire che i processi sono asincroni significa che ognuno di essi possiede una propria linea temporale indipendente. Ogni processo avanza per i fatti suoi.

## **non-determinism**

Significa che se esegui lo stesso identico programma concorrente dieci volte di seguito, con gli stessi identici dati di input, l'ordine esatto in cui i processi eseguiranno le loro istruzioni cambierà quasi sicuramente ogni volta.

programmi concorrenti sono non-deterministici. (Programma deterministico: dato un input, l'output è predefinito)

La caratteristica di speed independence definisce come dobbiamo progettare il software accettando questo asincronismo. Significa che:

- Nessuna assunzione sul tempo: Non puoi fare alcuna previsione o affidamento sulla velocità relativa dei processi. Non puoi sapere se il Processo A eseguirà 100 istruzioni nel tempo in cui il Processo B ne esegue una sola, o viceversa.
- Correttezza slegata dalle prestazioni: Un algoritmo concorrente si definisce corretto e speed-independent se il suo risultato finale è identico a prescindere da quanto vadano veloci o lenti i singoli flussi di controllo logici. Che la CPU sia lentissima, super veloce, o che un thread venga temporaneamente sospeso dal sistema operativo, l'output deve rimanere corretto.

In un sistema concorrente, ogni processo logico avanza alla propria velocità ed i processi sono totalmente asincroni. Non puoi mai sapere se il Processo A finirà prima del Processo B. La velocità di un processo può essere influenzata da mille fattori imprevedibili. Di conseguenza, non devi mai scrivere codice basandoti sull'idea che "visto che l'istruzione A è corta, finirà sicuramente prima dell'istruzione B".

# PROCESS INTERACTION

- **Process interaction**
  - any non trivial concurrent program is based on multiple processes that need to interact in some way
- **Basic kinds of interaction:** (interazioni ci sono quando ci sono dipendenze tra processi)
  - 1 – **cooperation**
  - 2 – **competition** / contention
  - 3 – **interferences** interazioni da evitare (esempio: corsa critica)



# PROCESS INTERACTION:

## 1 COOPERATION

- Refers to interactions which are both <sup>previste</sup> *expected* and <sup>desiderate</sup> *wanted*
  - they are part of the semantics of the concurrent program
- Two basic kinds
  - **communication**
    - concerns the need of realising an information flow among processes, typically realised in terms of messages
    - introduction of specific supports for the exchange of messages
  - **synchronisation** ordinamento temporale delle azioni di processi (es: un'azione di processo p avviene prima di un'altra azione di un processo q)
    - concerns the explicit definition or presence of **temporal relationships** or dependencies among processes and among actions of distinct processes
    - introduction of specific supports for the exchange of temporal signals

# PROCESS INTERACTION:

## 2 CONTENTION / COMPETITION

- Refers to interactions which are <sup>previste</sup> *expected* and *necessary*, but *not* <sup>desiderate</sup> *wanted* (gestire interazioni evitando interferenze)
  - typically concerns the need of coordinating the access by multiple processes to shared resources
- Two basic class of problems
  - **mutual exclusion** È la proprietà/regola che vieta a due processi di usare contemporaneamente la stessa risorsa fisica o logica (es. una variabile in memoria, un file, una stampante) per evitare che si corrompa.
    - <sup>regolamentare</sup> ~~ruling~~ the access to shared resources by distinct processes
  - **critical sections** È la porzione di programma che contiene le istruzioni che vanno a toccare quella risorsa condivisa. Regolamentare la sezione critica significa fare in modo che quel blocco di azioni venga eseguito da un solo processo alla volta, congelando gli altri pezzi di codice concorrenti che vorrebbero fare la stessa cosa.
    - <sup>regolamentare</sup> ~~ruling~~ the concurrent execution of blocks of actions by distinct processes

# SYNCHRONISATION VS. MUTUAL EXCLUSION

- Different - even if related - concepts
  - “synchronisation = mutual exclusion urban legend” [BUH-05]
    - false story, still present in textbooks / research papers
  - synchronisation defines a <sup>relazione temporale</sup>~~timing relationship~~ among processes
    - *maintaining time-relationships which includes actions happening at the same time or happening at the same relative <sup>ritmo</sup>~~rate~~ or simply some action having to occur before another (precedence relationships)*
  - mutual-exclusion defines a restriction on access to shared data
    - mutual-exclusion is meaningless if no shared data is involved
- Relationships
  - mutual-exclusion typically require some forms of implicit synchronisation
    - blocking some actions, waiting for other actions to complete
  - synchronisation does not necessarily require any kind of shared data and the mutual exclusion

# ON THE DIFFICULTY OF SYNCHRONISATION...

- “Alice and Bob live together, happily without cell-phones. Both are responsible to buy the milk when it finishes...”

| Time | Alice <small>Non appena finisce il latte, lo compro</small> | Bob                         |
|------|-------------------------------------------------------------|-----------------------------|
| 5:00 | Arrive home                                                 |                             |
| 5:05 | Look in the fridge; no milk                                 |                             |
| 5:10 | Leave for a grocery                                         |                             |
| 5:15 |                                                             | Arrive home                 |
| 5:20 |                                                             | Look in the fridge; no milk |
| 5:25 | <u>Buy milk</u>                                             | Leave for grocery           |
| 5:30 | <u>Arrive home; put milk in fridge</u>                      |                             |
| 5:40 |                                                             | <u>Buy milk</u>             |
| 5:45 |                                                             | <u>Arrive home; oh no!</u>  |

# A SOLUTION: NOTES IN THE FRIDGE (1/2)

- Looking for a solution to ensure that:
  - only one person buys the milk, when there is no milk
  - someone always buys the milk, when there is no milk
- Tentative solution: using notes on the fridge!

## PROGRAM for Alice & Bob

```
1 if (no note) then
2   if (no milk) then
3     leave note
4     buy milk
5     remove note
6   fi
7 fi
```

- “if you find that there is no milk and there is no note on the door of the fridge, then leave a note on the fridge’s door, go and buy milk, put the milk in the fridge, and remove your note.”
- Does it work? Not always actually...

# A SOLUTION: NOTES IN THE FRIDGE (2/2)

(..NOT SO EASY, ACTUALLY..)

Il processo non è atomico (è stato diviso in due azioni)

| Time | Alice                           | Bob                                |
|------|---------------------------------|------------------------------------|
| 5:00 | Arrive home                     |                                    |
| 5:05 | Look at the fridge; no note     |                                    |
| 5:10 | ...ops! need a toilet           |                                    |
| 5:15 | ...still at the toilet...       | Arrive home                        |
| 5:20 | ...still at the toilet...       | Look at the fridge; no note        |
| 5:21 | ...still at the toilet...       | Look in the fridge; no milk (argh) |
| 5:22 | ...still at the toilet...       | <u>leave note</u>                  |
| 5:25 | ...still at the toilet...       | go and buy milk                    |
| 5:45 | look in the fridge: no milk (*) | ...                                |
| 5:50 | <u>leave note...</u>            |                                    |

[\*] Alice does not realize that a note was put on the fridge (she is not really a good observer) and strictly follows the established program

# PROCESS INTERACTION:

3

## INTERFERENCES

Interazioni da evitare

non sono né previste né desiderate

- Refers to interactions which are ~~neither expected, nor wanted~~
  - producing bad effects only when the ~~ratio among~~ the process speeds assumes specific values (time-dependent errors)
- The “nightmare” of concurrent programming
  - “heisen-bugs” → è un bug che sembra scomparire o modificare il suo comportamento quando si tenta di studiarlo, analizzarlo o correggerlo.
    - when debugging influence the bugs...

# INTERFERENCES: RACE CONDITIONS

Situazione in cui, a seconda dell'ordine con cui i thread eseguono azioni, i risultati possono essere diversi da quelli corretti che ci aspettiamo

- **Race condition** or **race hazard** or simply **race**  
si verifica ogni volta che
  - ~~whenever~~ two or more processes concurrently access and update shared resources, and the result of the single update *depends on the specific order occurring in process access*
- Related to two main types of programming errors
  - bad management of expected interactions
  - presence of spurious interactions not expected in the problem



# CRITICAL SITUATIONS

Se ho Deadlock, ho Starvation,  
ma non viceversa

- Interferences and errors in concurrent programs can lead to *critical situations* for the concurrent system in the overall
  - A – **Deadlock** (...or deadly embrace (Dijkstra)) riguardante un insieme di processi
  - B – **Starvation** (or unfairness) riguardante un singolo processo
  - C – **Livelock** simile a Deadlock, ma i processi non sono bloccati e non riescono proseguire

A

# DEADLOCK

- Situation <sup>in cui</sup> wherein two or more <sup>azioni/processi che sono in competizione tra loro</sup> competing actions (processes) are waiting for the other to finish, <sup>e di conseguenza nessuna delle due viene mai portata a termine.</sup> and thus neither ever does
  - typically concerns the release of a locked shared resource, the reception of a temporal signal or a message
- Remark
  - it concerns *multiple processes*

# B STARVATION

- Situation <sup>in cui</sup> wherein a process <sup>rimane bloccato in un'attesa infinita</sup> is blocked in an infinite waiting
- Resource starvation
  - the process <sup>costantemente</sup> is perpetually denied in accessing necessary resources.
  - without those resources, the program can never finish its task
- Remark
  - it concerns a <sup>single</sup> process



# LIVELOCK

Il Livelock è una situazione paradossale dove i processi sono "vivi" (si muovono, consumano CPU), ma il sistema nel suo complesso è completamente bloccato. Mentre nel Deadlock i processi sono come statue (fermi, in attesa di una risorsa che non arriverà mai), nel Livelock i processi sono iperattivi, ma girano a vuoto.

- A livelock is similar to a deadlock, <sup>con la differenza che</sup> ~~except that the states of the processes involved in the livelock constantly change with regard to one another, none progressing~~ <sup>cambiano continuamente l'uno rispetto all'altro</sup>  
<sup>senza che nessuno di essi proceda</sup>
- Livelock is a special case of resource starvation
  - the general definition only states that a specific process is not progressing



Immagina di camminare in un corridoio stretto e di trovarti di fronte a un'altra persona che cammina in senso opposto.

- Situazione di Deadlock: Entrambi vi fermate e nessuno dei due si sposta. Rimanete immobili aspettando che l'altro faccia il primo passo. Siete bloccati e statici.

- Situazione di Livelock: Tu ti sposti a sinistra per lasciarlo passare, e lui simultaneamente si sposta alla sua destra (quindi dalla stessa parte) per lasciarti passare. Allora tu torni a destra, e lui torna a sinistra. Continuate a muoverti freneticamente da una parte all'altra, ma rimanete sempre uno di fronte all'altro. State facendo un sacco di movimenti, ma non state progredendo nel corridoio.

# CONCURRENT LANGUAGES AND MACHINES

Java è un linguaggio ibrido: non è un linguaggio concorrente puro

- To *describe / specify* a concurrent program we need **concurrent programming languages**
  - enabling programmers to write down programs as set of instructions to be executed concurrently
- To *execute* a concurrent program we need a **concurrent machine** <sup>(può essere logica/virtuale)</sup>
  - a machine (which can be abstract) designed to handle the execution of multiple sequential processes, <sup>sfruttando</sup> ~~by exploiting~~ multiple processors (physical or virtual)

# CONCURRENT MACHINES

Non si riferisce necessariamente a un computer fisico con tanti core, ma a un ambiente di esecuzione (composto da Hardware + Sistema Operativo + Runtime del linguaggio) che permette al programmatore di scrivere codice come se avesse risorse infinite a disposizione.

- A **concurrent machine** <sup>fornisce</sup> ~~provides:~~
  - a support for the execution of concurrent programs **and realizing** ~~then~~ concurrent computations
  - as many virtual processors as the number of processes <sup>che compongono</sup> ~~composing~~ the concurrent computation
- Providing basic mechanisms for:
  - **multiprogramming**
    - virtual processors generation and management
  - **synchronisation and communication**
  - **access control** to resources

Significa che se scrivi un programma con 100 attività indipendenti, la macchina concorrente ti permette di ragionare come se avessi 100 piccoli computer dedicati. Magari il tuo PC ha solo 4 core reali. La macchina concorrente usa il Multiprogramming per "affettare" il tempo dei 4 core e distribuirlo ai 100 processi virtuali così velocemente che sembrano girare tutti insieme.

Dato che molti processi virtuali lottano per poche risorse reali (RAM, Disco, Stampante), la macchina deve fare da "vigile urbano". Deve impedire che due processi scrivano sullo stesso file contemporaneamente (evitando le Race Conditions di cui parlavamo prima).

# BASIC MECHANISMS

- **Multiprogramming**

- set of mechanisms that make it possible to create new virtual processors and allocate physical processors of the lower-level machine to the virtual processors by means of scheduling algorithms

- **Synchronisation and Communication**

Dato che i processi vivono nello stesso "ambiente", devono potersi parlare (possibilità di cooperazione).

- two different typologies of mechanisms, related to two different *architectural models* for concurrent machines:
  - **shared memory model**
    - presence of a shared memory among the virtual processors
    - example: multi-threaded programming
  - **message passing model**
    - every virtual processor has its own memory and no shared memory among processors is present
    - every communication and interaction among processors is realized through message passing

# FROM MACHINES TO PROGRAMMING LANGUAGES

- Programming languages for specifying concurrent programs on top of concurrent machines
  - programs organised as sets of sequential processes to be executed concurrently on the virtual processors of the concurrent machine
- Basic constructs for
  - **specifying concurrency**
    - creation of multiple processes
  - **specifying process interaction**
    - synchronisation and communication
    - mutual exclusion



# CONCURRENT PROGRAMMING LANGUAGES - DESIGN APPROACHES

- Three main design approaches
  - **sequential language + library with concurrent primitives**
    - e.g. C + PThreads
  - **language designed for concurrency**
    - e.g. OCCAM, ADA, Erlang, Go
  - **hybrid approach**
    - sequential paradigm extended with a native support for concurrency
      - e.g. Java, Scala
    - library and patterns based on basic mechanisms
      - e.g. java.util.concurrent

# CONCURRENCY IN MAINSTREAM LANGUAGES

- For the most part, mainstream languages - both procedural (like C) and Object-Oriented (Java) - provide a support for the creation and execution of processes by means of libraries
  - without extending the language
  - not completely true for Java
- Support for multi-threaded programming
  - threads as implementation of the abstract notion of process
    - also called “lightweight processes” by referring to OS “heavyweight processes”
  - not to be confused with the notion of process as defined in OS
    - process as a programming in execution, with one or multiple control flows (threads)
- Main examples
  - multi-threaded programming in Java
  - Pthread library for C/C++ language on POSIX systems

# BEYOND THREADS: RICH SCENARIO

- **Task-based frameworks**
  - e.g. Java Executor framework
- **Asynchronous Programming**
  - event-loop based models
  - AsyncTasks everywhere
- **Actors**
  - Concurrent OOP
  - *active objects* variant
- **Synchronous Message Passing**
  - Processes + synchronous message passing
- **Reactive Programming**
  - e.g. Functional Reactive Programming, Reactive extensions (Rx).
- **Parallel computing**
  - GPGPU programming
  - Lambda Architecture / Parallelism for BigData

# BIBLIOGRAPHY

- **[SUT-12]**
  - Sutter's Mill. Herb Sutter on software, hardware, and concurrency. "Welcome to the Jungle". <http://herbsutter.com/welcome-to-the-jungle/>
- **[AND-83]**
  - Gregory Andrews and Fred Schneider - "Concepts and Notations for Concurrent Programming", *ACM Computing Surveys*, Vol. 15, No. 1, March 1983
- **[CLE-96]**
  - Rance Cleaveland, Scott Smolka et al - "Strategic Directions in concurrency Research", *ACM Computing Surveys*, Vol. 28, No. 4, Dec. 1996
- **[ROS-97]**
  - Roscoe, A. W. (1997). *The Theory and Practice of Concurrency*. Prentice Hall. ISBN 0-13-674409-5.
- **[BUH-05]**
  - Peter Buhr and Ashif Harji. "Concurrent Urban Legends". *Concurrency and Computation: Practice and Experience*. 2005. 17:1133-1172.